REPUBLIQUE DU TCHAD
Unité – Travail – Progrès
****
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
****
UNIVERSITE DE MOUNDOU
****
FACULTE DES SCIENCES EXACTES ET APPLIQEES
****
DEPARTEMENT DES SCIENCES TECHNOLOGIES
BP 206 Moundou, Tchad
Tel: (00235) 65291329
Email: udm@univ-mdou.org
****

جمهورية تشاد
****
وحدة – عمل – تقد
****
وزارة التعليم العالي
****
جامعة مو ندو
****
كلية العلوم الدقيقة والتطبيقية
****
قسم علوم التكنولوجيا

RAPPORT DU TRAVAUX PRATIQUE N° 03

THEME :

Conversion de l'application web « Gestion des commandes » (Laravel) en application mobile Android avec Flutter

Conversion mobile réalisée par :

- ✓ **ALI Adoum Abakar**
- ✓ **CHAHAD Banane Moustapha**

Encadrant : **Prof. Arouna NDAM NJOYA**

Date : **Octobre 2026**

Sommaire

1. Introduction 2
1.1 Contexte 2
1.2 Objectifs 2
1.3 Contraintes 2
1.4 Démarche suivie 2
2. Analyse du projet web existant 3
2.1 Présentation technique 3
2.2 Fonctionnalités 3
2.3 Modèle de données 3
2.4 Routes et contrôleurs 4
2.5 Règles de gestion relevées 5
3. Conception de la solution mobile 6
3.1 Choix technologiques 6
3.2 Architecture générale 7
3.3 Correspondance Laravel → Flutter 8
3.4 Organisation du code 8
4. Réalisation 10
4.1 Base de données locale 10
4.2 Gestion métier et transactions 10
4.3 Authentification 11
4.4 Interface utilisateur 12
4.5 Navigation 14
4.6 Tableau de bord 16
5. Tests et validation 17
5.1 Stratégie 17
5.2 Recette manuelle 17
5.3 Contrôles propres à Android 18
6. Build et déploiement Android 19
6.1 Préparation 19
6.2 APK de test ou de remise 19
6.3 Signature et publication sur Google Play 19
7. Écarts, limites et perspectives 20
7.1 Écarts avec la version web 20
7.2 Limites 20
7.3 Perspectives 20
8. Conclusion 21
Annexe A. Liste des fichiers du projet Flutter 22
Annexe B. Commandes utiles 24

1. Introduction

1.1 Contexte

Le TP Examen « Gestion des commandes » est une application web de gestion des **produits**, des **clients** et des **commandes**, complétée par un **tableau de bord** statistique. Elle a été réalisée avec **Laravel 12**, **PHP 8.2+** et **MySQL** par l'équipe composée de CHAHAD Banane Moustapha et ALI Adoum Abakar, sous l'encadrement du Prof. Arouna NDAM NJOYA.

Ce rapport présente la conversion de cette application web en **application mobile Android autonome**, développée uniquement avec **Flutter**. Il n'y a plus de serveur PHP ni de base MySQL : les données sont enregistrées localement sur le téléphone et l'application fonctionne sans connexion Internet.

1.2 Objectifs

- Reprendre **toutes les fonctionnalités** du site web : authentification, produits, clients, commandes, lignes de commande et tableau de bord.
- Conserver l'**identité visuelle** de l'interface Bootstrap : couleurs, espacements, tableaux, boutons, menu latéral.
- Remplacer le backend Laravel par une architecture Flutter locale : base **SQLite**, repositories et session sécurisée.
- Garantir la **cohérence des stocks** grâce à des transactions (tout ou rien).
- Livrer un code **propre, commenté, testé et maintenable**, ainsi qu'un APK installable.

1.3 Contraintes

- Flutter uniquement : **aucun** Laravel ni backend PHP.
- Interface visuelle et fonctionnalités **identiques** à la version web.
- Plateforme cible : **Android uniquement** (pas de version iOS).
- Code organisé en couches, commenté et facile à faire évoluer.

1.4 Démarche suivie

1. **Analyse** du projet web : routes, contrôleurs, modèles Eloquent, migrations et vues Blade.
2. **Conception** de l'architecture Flutter et de la correspondance Laravel → Flutter.
3. **Réalisation** : base locale, logique métier, authentification, interface, navigation, tableau de bord.
4. **Tests** : tests unitaires des règles de stock, recette manuelle, contrôles sur Android.
5. **Build** de l'APK et préparation de la publication.

2. Analyse du projet web existant

2.1 Présentation technique

L'application web suit le modèle **MVC** de Laravel. Le tableau ci-dessous résume la pile technique relevée dans le code source.

Tableau 1 : Pile technique de la version web

Élément	Technologie
Langage et framework	PHP 8.2+ / Laravel 12
Base de données	MySQL / MariaDB (WAMP ou XAMPP)
Interface	17 vues Blade, Bootstrap 5.3, Chart.js
Organisation	5 contrôleurs (Auth, Produit, Client, Commande, Dashboard) et 6 modèles Eloquent
Authentification	Authentification Laravel ; connexion limitée à 5 tentatives par minute

2.2 Fonctionnalités

Tableau 2 : Fonctionnalités de l'application et écrans mobiles correspondants

Module	Actions disponibles	Écrans Flutter
Authentification	Inscription, connexion (« Se souvenir de moi »), déconnexion	LoginPage, RegisterPage
Produits	Liste, ajout, modification	ProduitsPage, ProduitFormPage
Clients	Liste, ajout, modification, avec département et région saisis librement et créés au besoin	ClientsPage, ClientFormPage
Commandes	Liste, création, modification, détail, suppression	CommandesPage, CommandeFormPage, CommandeDetailPage
Lignes de commande	Ajout d'un produit, modification de la quantité, retrait d'un produit	CommandeDetailPage, LigneEditPage
Tableau de bord	Chiffre d'affaires total, CA par mois, produits les plus vendus, CA par département, meilleurs clients, produits à stock faible	DashboardPage

2.3 Modèle de données

Les migrations Laravel définissent sept tables utiles. Les produits et les clients sont reliés par la table d'association **ligneCommande**, qui porte la quantité commandée. Les clients sont localisés par une **région**, elle-même rattachée à un **département**. La table **users** est indépendante : elle sert uniquement à l'authentification.

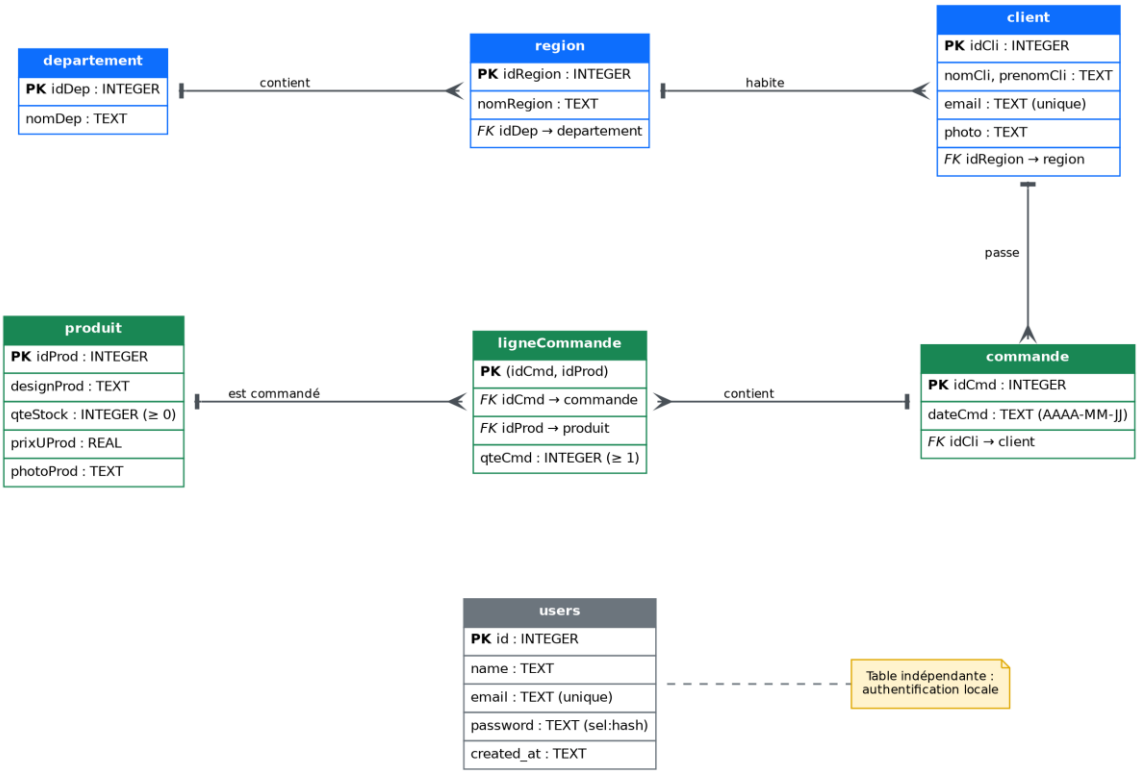


Figure 1 : Modèle de données (tables, clés primaires et clés étrangères)

Tableau 3 : Description des tables

Table	Rôle	Clés et contraintes
users	Comptes d'accès à l'application	PK id ; email unique ; mot de passe haché
departement	Niveau géographique supérieur	PK idDep
region	Région rattachée à un département	PK idRegion ; FK idDep (RESTRICT)
client	Clients de l'entreprise	PK idCli ; email unique ; FK idRegion (RESTRICT)
produit	Catalogue et stock	PK idProd ; prixUProd ; contrainte qteStock ≥ 0
commande	En-tête d'une commande	PK idCmd ; index sur dateCmd ; FK idCli (RESTRICT)
ligneCommande	Produits d'une commande	PK composée (idCmd, idProd) ; contrainte qteCmd ≥ 1 ; FK idCmd (CASCADE) ; FK idProd (RESTRICT)

> **Remarque.** Les colonnes `photo` (client) et `photoProd` (produit) existent dans la base mais ne sont utilisées par aucune vue Blade. Elles ont été conservées dans le schéma mobile pour rester compatible avec le modèle d'origine.

2.4 Routes et contrôleurs

Le fichier `routes/web.php` sépare deux groupes : les pages **invité** (connexion, inscription) et les pages **authentifiées**. Chaque route est associée à un écran Flutter.

Tableau 4 : Routes web et écrans mobiles correspondants

Route web	Contrôleur / action	Écran mobile (route)
/login, /register	AuthController (connexion, inscription)	LoginPage, RegisterPage (/login, /register)
POST /logout	AuthController::logout	Bouton « Déconnexion » du menu
/	DashboardController::index	DashboardPage (/)
/produits	ProduitController::index	ProduitsPage (/produits)
/produits/create · /produits/{id}/edit	ProduitController::create, store, edit, update	ProduitFormPage (/produits/create, /produits/:id/edit)
/clients	ClientController::index	ClientsPage (/clients)
/clients/create · /clients/{id}/edit	ClientController::create, store, edit, update	ClientFormPage (/clients/create, /clients/:id/edit)
/commandes	CommandeController::index	CommandesPage (/commandes)
/commandes/create	CommandeController::create, store	CommandeFormPage (/commandes/create)
/commandes/{id}	CommandeController::show	CommandeDetailPage (/commandes/:id)
/commandes/{id}/edit	CommandeController::edit, update	CommandeFormPage (/commandes/:id/edit)
DELETE /commandes/{id}	CommandeController::destroy	Bouton « Supprimer » avec boîte de confirmation
POST /commandes/{id}/produits	CommandeController::ajouterProduit	Formulaire « Ajouter un produit » du détail
/commandes/{id}/produits/{p}/edit	CommandeController::editerProduit, modifierProduit	LigneEditPage (/commandes/:id/produits/:idProd/edit)
DELETE /commandes/{id}/produits/{p}	CommandeController::retirerProduit	Bouton « Retirer » de LigneEditPage

2.5 Règles de gestion relevées

- **R1.** Un produit ne peut figurer qu'une seule fois dans une commande (clé primaire composée).
- **R2.** La quantité commandée est supérieure ou égale à 1 ; le stock ne peut pas être négatif.
- **R3.** Ajouter un produit à une commande **diminue le stock** ; l'ajout est refusé si la quantité demandée dépasse le stock.
- **R4.** Modifier une quantité **ajuste le stock de la différence** ; la quantité maximale possible est le stock disponible plus la quantité déjà réservée.
- **R5.** Retirer un produit ou supprimer une commande **remet les quantités en stock**.
- **R6.** La modification globale d'une commande est **transactionnelle** : tout est enregistré, ou rien.
- **R7.** L'e-mail d'un client est unique ; département et région sont créés s'ils n'existent pas encore.
- **R8.** Les pages sont réservées aux utilisateurs connectés ; le message d'erreur de connexion est identique pour un e-mail inconnu et un mot de passe incorrect ; le mot de passe compte au moins 8 caractères.
- **R9.** Les montants s'affichent en FCFA avec séparateur d'espace (« 12 500 FCFA ») ; un produit est en **stock faible** sous le seuil de 20.

3. Conception de la solution mobile

3.1 Choix technologiques

Le choix principal est de conserver un **modèle relationnel** : le schéma MySQL se transpose presque tel quel en SQLite, et les statistiques du tableau de bord (jointures, `SUM`, `GROUP BY`) restent des requêtes SQL.

Tableau 5 : Technologies retenues

Besoin	Choix	Justification
Framework	Flutter (Dart), Android seul	Contrainte du projet : application mobile sans backend PHP
Base locale	sqflite + path	Schéma relationnel, clés étrangères, transactions, fonctionnement hors ligne
État	provider	Simple et suffisant pour partager l'état de connexion
Navigation	go_router	Routes déclaratives et redirection selon la connexion (équivalent des middlewares)
Session	flutter_secure_storage	Stockage chiffré par le Keystore Android
Mot de passe	crypto (SHA-256 + sel)	Hachage sans serveur
Formats	intl + flutter_localizations	Dates jj/mm/aaaa, nombres « 12 500 », sélecteur de date en français
Graphiques	fl_chart	Équivalent de Chart.js : courbe, donut, barres
Tests	flutter_test + sqflite_common_ffi	Base SQLite en mémoire exécutable sur PC
Alternatives écartées	Firebase, Supabase ; Hive, Isar	Backend cloud hors contrainte « Flutter seul » ; absence de jointures

3.2 Architecture générale

La figure 2 met en regard l'architecture web et l'architecture mobile : chaque brique Laravel a un équivalent Flutter. Les contrôleurs et modèles Eloquent deviennent des **repositories** qui accèdent directement à SQLite.

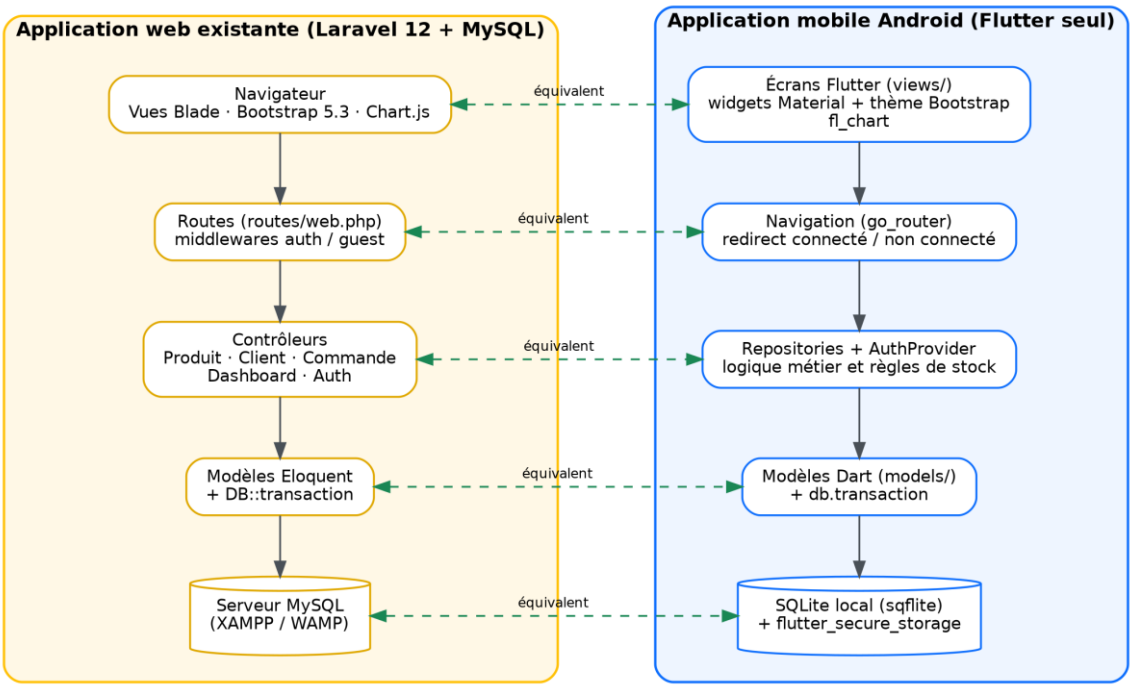


Figure 2 : Architecture web (Laravel + MySQL) et architecture mobile (Flutter + SQLite)

L'application mobile est organisée en **cinq couches** qui ne communiquent que de haut en bas. Les écrans n'accèdent jamais directement à SQLite : ils passent par un repository, comme les vues Blade passent par les contrôleurs.

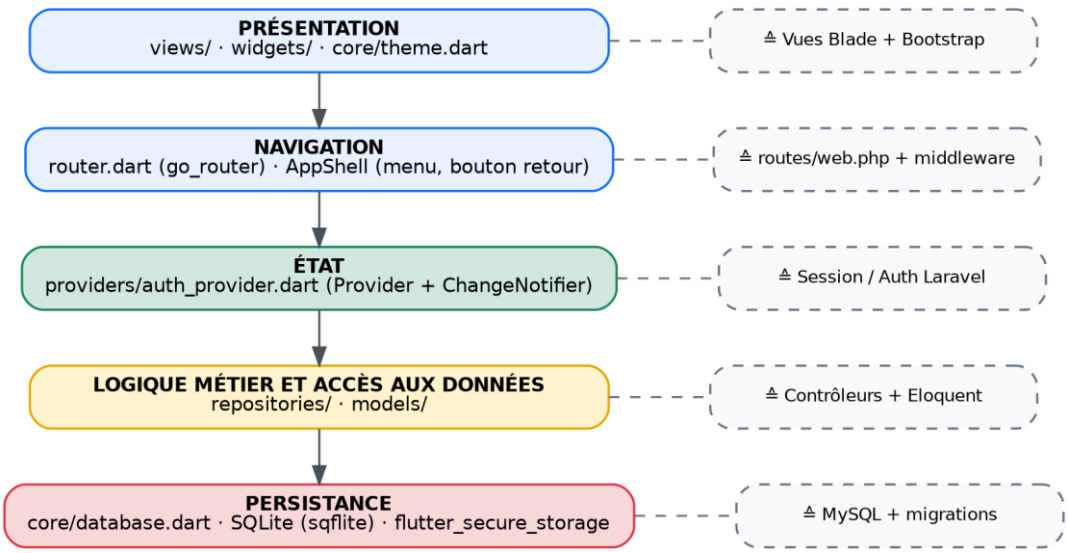


Figure 3 : Architecture en couches de l'application Flutter

3.3 Correspondance Laravel → Flutter

Tableau 6 : Équivalences entre les concepts Laravel et Flutter

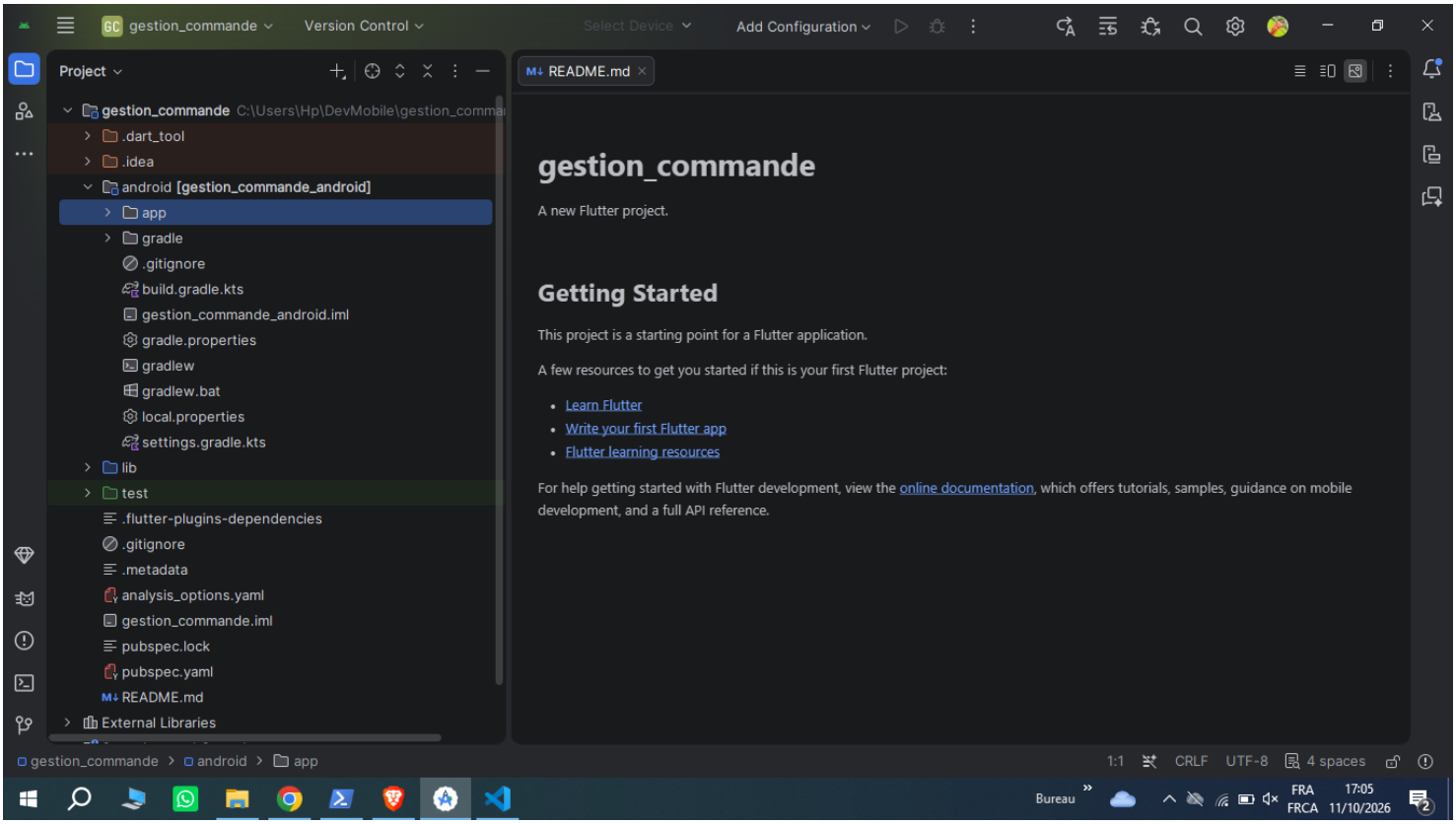
Concept	Laravel	Flutter
Schéma de base	Migrations	AppDatabase._createSchema et _upgrade (version du schéma)
Données de démonstration	Seeders	AppDatabase._seed
Accès aux données	Modèles Eloquent	models/*.dart + repositories/*.dart
Logique métier	Contrôleurs	repositories (règles de stock, validations métier)
Transactions	DB::transaction	db.transaction (sqflite)
Vues	Blade + Bootstrap	views/ + widgets/ + core/theme.dart
Protection des pages	middleware auth / guest	redirect de go_router
Connexion	Auth::attempt	AuthRepository.login
Session	Session / cookie	flutter_secure_storage
Validation	Règles de validation Laravel	validator des TextFormField (mêmes messages en français)
Messages flash	session('success')	SnackBar (fonction toast)
Confirmation	confirm() JavaScript	AlertDialog
Graphiques	Chart.js	fl_chart
Formatage	number_format, format('d/m/Y')	core/format.dart (nombre, fcfa, dateFr)
Responsive	@media (min-width: 992px)	MediaQuery.sizeOf(context).width >= 992

3.4 Organisation du code

Le projet Flutter `gestion_commande` comporte **35 fichiers** (voir l'annexe A). Les dossiers reprennent la séparation des responsabilités de Laravel.

```
lib/
├── main.dart            point d'entrée : base, session, thème, langue française
├── router.dart          routes + garde d'authentification
├── core/                theme · format · ui · database
├── models/              produit · client · commande · ligne_commande
├── repositories/        auth · produit · client · commande · dashboard
├── providers/           auth_provider
├── widgets/             app_shell · bs_button · bs_card · bs_table · page_header
│                        stat_card · auth_card
└── views/               auth · dashboard · produits · clients · commandes
test/
└── commande_repository_test.dart
```

Le projet a été créé avec Android Studio pour la seule plateforme Android (figure 4).



4. Réalisation

4.1 Base de données locale

La classe `AppDatabase` ouvre la base SQLite `ligne_commandes.db` dans l'espace privé de l'application. Elle remplace MySQL et les migrations :

- `onConfigure` active les **clés étrangères** (`PRAGMA foreign_keys = ON`), désactivées par défaut dans SQLite.
- `onCreate` crée les 7 tables puis insère un **jeu de données de démonstration** : 3 départements et régions, 3 clients, 4 produits et 6 commandes réparties d'août à octobre 2026.
- `onUpgrade` accueillera les futures migrations : on incrémente le numéro de version du schéma et on ajoute l'instruction `ALTER TABLE` correspondante.

> **Point d'attention.** `onCreate` ne s'exécute qu'**une seule fois**, à la création du fichier de base. Après un changement du schéma ou des données de démonstration, il faut désinstaller l'application du téléphone pour recréer la base.

4.2 Gestion métier et transactions

Toutes les règles de stock (R3 à R6) sont implémentées dans `CommandeRepository`. Chaque opération qui touche à la fois une ligne de commande et un stock s'exécute dans une **transaction** : si une erreur est détectée, SQLite annule toutes les modifications.

Tableau 7 : Règles de gestion du stock

Opération	Effet sur le stock	Contrôles et erreurs
Ajouter un produit (quantité q)	stock − q	Refus si le produit est déjà dans la commande ; refus si q > stock
Modifier la quantité (de a vers n)	stock − (n − a)	Refus si n − a > stock, soit n > stock + a (quantité maximale)
Retirer un produit	stock + quantité de la ligne	Refus si la ligne n'existe pas
Supprimer la commande	stock + quantité de chaque ligne	Lignes supprimées en cascade
Enregistrer le formulaire de modification	Combinaison des cas précédents	Transaction unique : en cas d'erreur, rien n'est enregistré

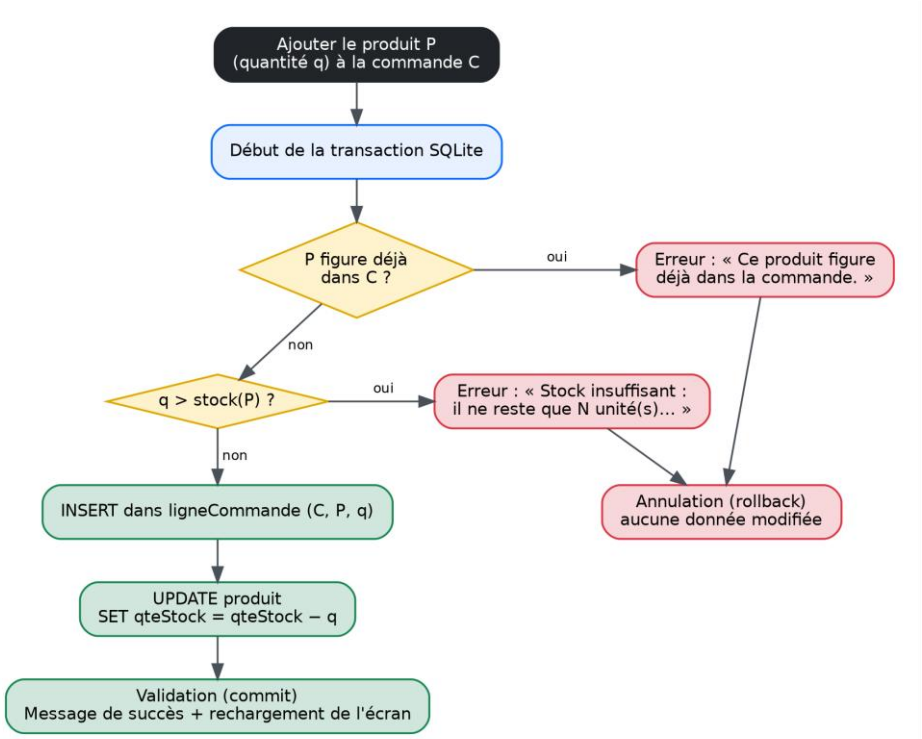


Figure 5 : Déroulement de l'ajout d'un produit à une commande

Extrait du code correspondant :

```
Future<void> ajouterProduit(int idCmd, int idProd, int qte) async {
  final db = await AppDatabase.instance;
  await db.transaction((tx) async {
    final p = (await tx.query('produit', where: 'idProd = ?', whereArgs: [idProd])).first;
    final stock = p['qteStock'] as int;

    final dejaLa = await tx.query('ligneCommande',
        where: 'idCmd = ? AND idProd = ?', whereArgs: [idCmd, idProd]);
    if (dejaLa.isNotEmpty) throw ErreurMetier('Ce produit figure déjà dans la commande.');
    if (qte > stock) {
      throw ErreurMetier('Stock insuffisant : il ne reste que $stock unité(s)…');
    }

    await tx.insert('ligneCommande', {'idCmd': idCmd, 'idProd': idProd, 'qteCmd': qte});
    await tx.rawUpdate('UPDATE produit SET qteStock = qteStock - ? WHERE idProd = ?',
        [qte, idProd]);
  });
}
```

Les erreurs de gestion sont levées sous la forme d'une exception `ErreurMetier` dont le message, en français, est affiché tel quel à l'utilisateur dans un bloc d'alerte rouge.

4.3 Authentification

Sans serveur, l'authentification repose sur la table locale `users`. Le mot de passe n'est jamais stocké en clair : il est haché avec **SHA-256** et un **sel aléatoire** de 16 octets, puis enregistré sous la forme `sel:hash`. L'identifiant de l'utilisateur connecté est conservé par `flutter_secure_storage`, qui s'appuie sur le Keystore Android.

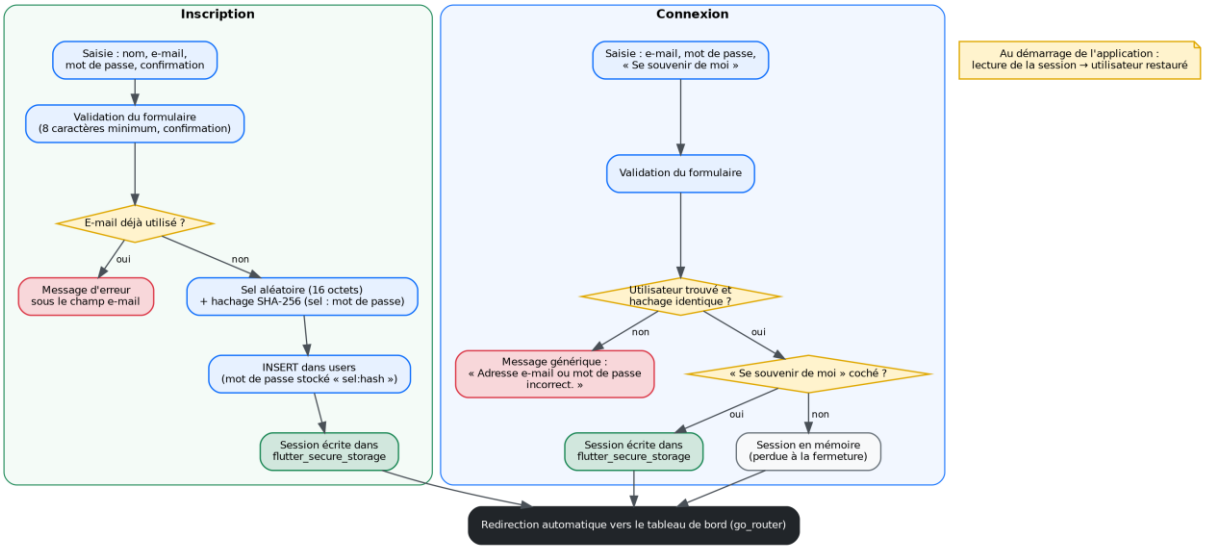


Figure 6 : Déroulement de l'inscription et de la connexion

- **Messages identiques au web** : « Adresse e-mail ou mot de passe incorrect. » pour tout échec de connexion, afin de ne pas révéler si un e-mail existe.
- **« Se souvenir de moi »** : la session n'est écrite dans le stockage sécurisé que si la case est cochée.
- **Restauration au démarrage** : `AuthProvider.init` relit la session avant l'affichage du premier écran.
- **Sauvegarde Android désactivée** (`android:allowBackup="false"`) pour éviter qu'une session chiffrée soit restaurée sans sa clé.

4.4 Interface utilisateur

Le thème `core/theme.dart` reprend les couleurs Bootstrap 5.3 : bleu primaire #0D6EFD, succès #198754, danger #DC3545, avertissement #FFC107, sombre #212529 (menu et en-têtes de tableau), fond de page #F8F9FA. Des composants réutilisables reproduisent les classes Bootstrap utilisées dans les vues Blade.

Tableau 8 : Composants d'interface réutilisables

Composant Flutter	Équivalent web	Rôle
Bs (theme.dart)	Variables Bootstrap	Palette, champs, cartes, boutons
AppShell	layouts/app.blade.php	Sidebar fixe de 270 px à partir de 992 dp, menu latéral (Drawer) sinon ; gestion du bouton retour Android
AuthCard	layouts/auth.blade.php	Fond dégradé et carte centrée (connexion, inscription)
BsCard	.card, .card-header	Carte avec en-tête optionnel
BsButton	.btn, .btn-sm, .btn-outline-*	Boutons pleins ou à contour
BsTable	.table-dark, .table-striped, .table-responsive	En-tête sombre, lignes alternées, défilement horizontal
StatCard	Cartes du dashboard	Bordure colorée, valeur et unité
PageHeader	d-flex justify-content-between	Titre de page et bouton d'action
AlertBox, toast()	.alert-*, messages flash	Erreurs de formulaire, messages de succès

Le comportement **responsive** suit le même seuil que le site web (992 px). En dessous, le menu devient un tiroir ouvert par le bouton hamburger ; au-dessus (tablette, paysage large), la sidebar est fixe. Les tableaux défilent horizontalement si l'écran est trop étroit, comme `.table-responsive`.

Les figures 7 et 8 présentent des **maquettes schématiques** des principaux écrans, dessinées avec les couleurs et la disposition du thème. Elles illustrent la structure des écrans et ne sont pas des captures de l'application.

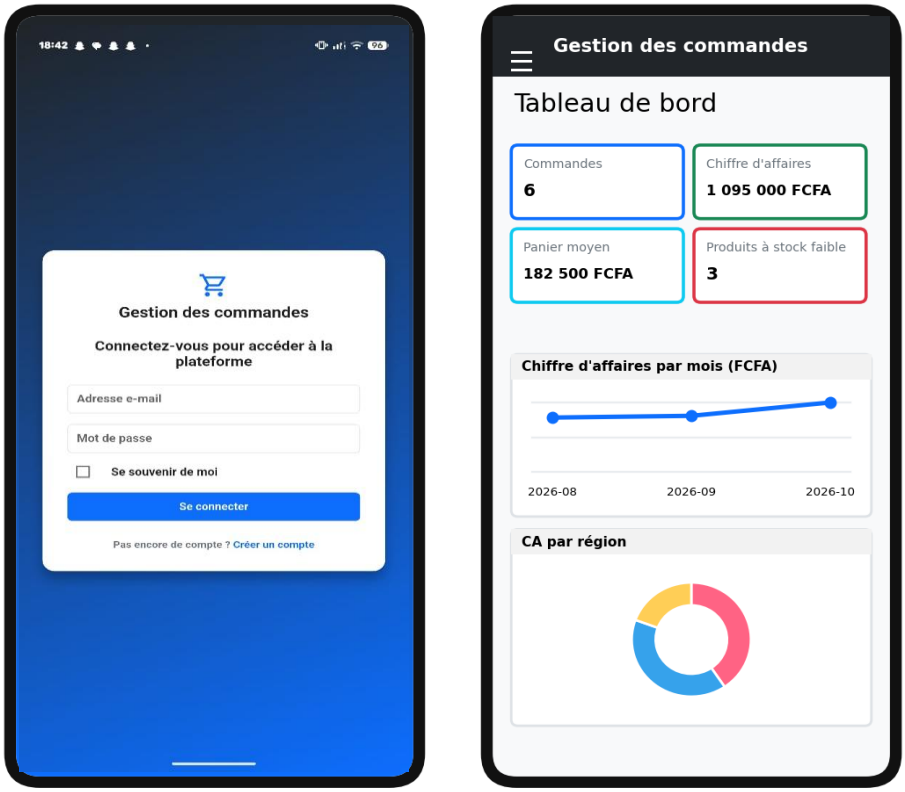


Figure 7 : Maquettes des écrans de connexion et du tableau de bord

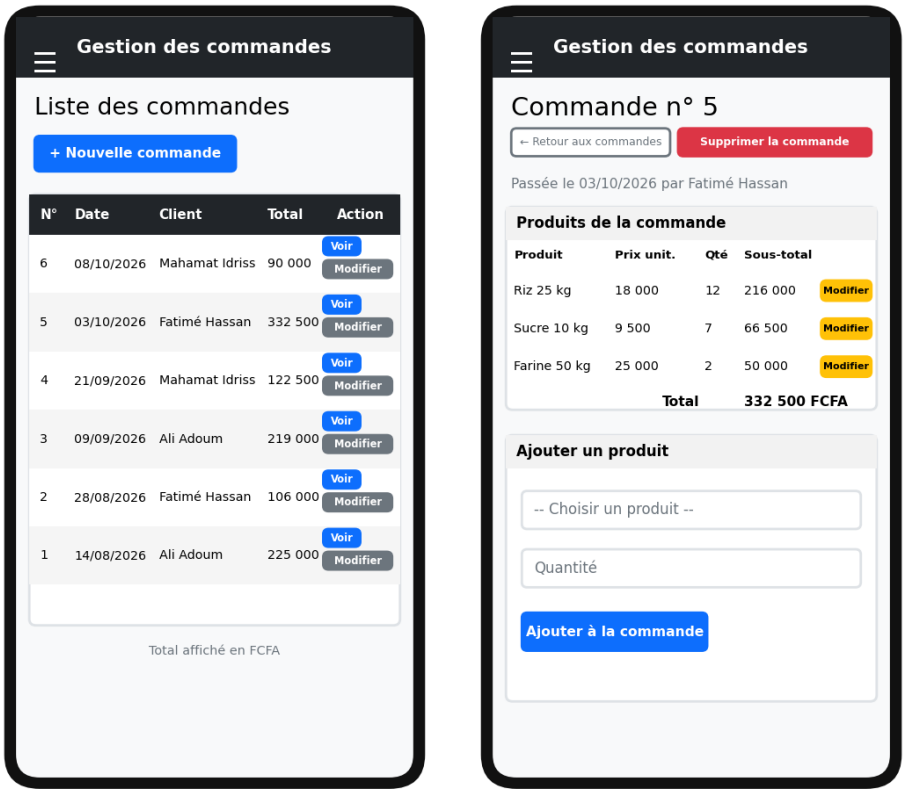


Figure 8 : Maquettes des écrans « Liste des commandes » et « Détail d'une commande »

4.5 Navigation

`go_router` déclare les routes de l'application. Sa fonction `redirect` joue le rôle des middlewares `auth` et `guest` : un utilisateur non connecté est renvoyé vers `/login`, un utilisateur connecté ne peut plus ouvrir `/login` ni `/register`.

```
redirect: (context, state) {
  if (!auth.ready) return null;
  final onAuthPage = state.matchedLocation == '/login' ||
                     state.matchedLocation == '/register';
  if (!auth.isLoggedIn && !onAuthPage) return '/login';
  if (auth.isLoggedIn && onAuthPage) return '/';
  return null;
},
```

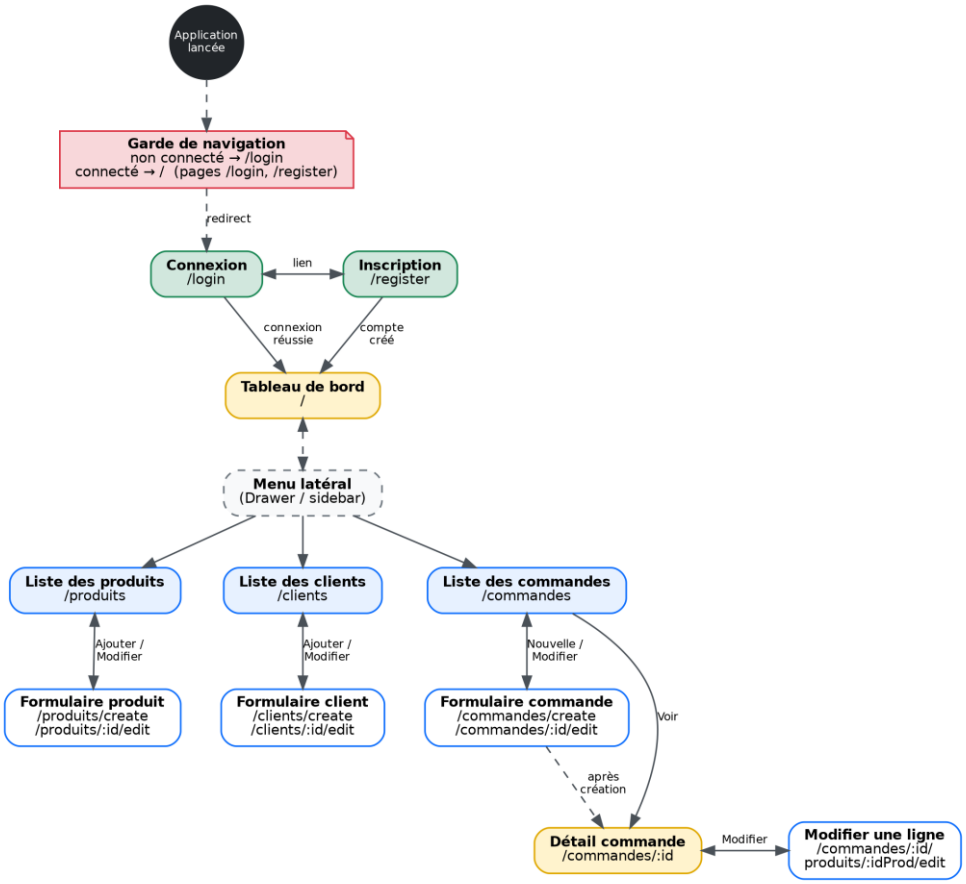


Figure 9 : Carte de navigation de l'application

Trois comportements spécifiques à Android ont été traités :

- **Bouton retour** : depuis une section (Clients, Produits…), il ramène au tableau de bord ; depuis le tableau de bord, il quitte l'application. Les formulaires, ouverts par `push`, reviennent à la liste d'origine.
- **Rechargement des listes** : chaque liste attend le retour de l'écran ouvert, puis se recharge pour afficher les modifications.
- **Création d'une commande** : le formulaire renvoie l'identifiant de la nouvelle commande, et la liste ouvre aussitôt son détail pour y ajouter des produits.

4.6 Tableau de bord

Le tableau de bord est calculé par `DashboardRepository.charger()`, qui exécute **8 requêtes SQL** (comptages, sommes, regroupements et jointures) et renvoie un objet `DashboardData`. La page affiche aussi le nombre de requêtes et la durée du calcul, comme la version web.

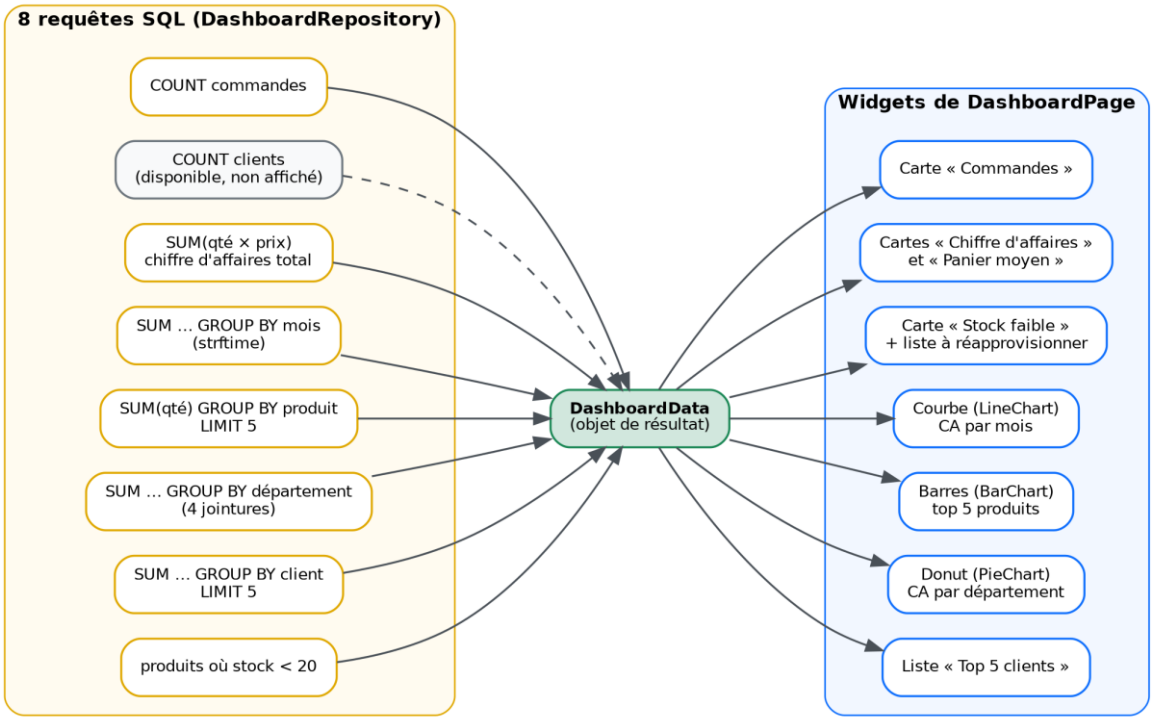


Figure 10 : Des requêtes SQL aux widgets du tableau de bord

Le montant d'une ligne est calculé par `quantité × prix unitaire`. Avec les données de démonstration, le tableau de bord affiche **6 commandes**, un chiffre d'affaires de **1 095 000 FCFA**, un panier moyen de **182 500 FCFA** et **3 produits en stock faible** (huile : 10, sucre : 15, farine : 15). La figure 11 illustre ces statistiques.

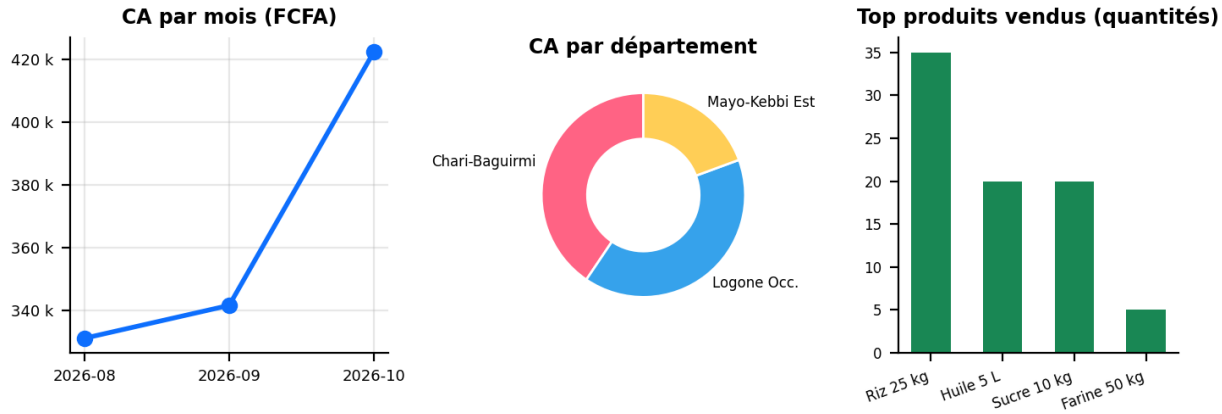


Figure 11 : Illustration des statistiques calculées sur les données de démonstration (graphiques générés avec Matplotlib)

> **Remarque.** Sans commande contenant au moins un produit, le tableau de bord affiche des zéros et le message « Aucune donnée. » : les statistiques sont calculées à partir des lignes de commande.

5. Tests et validation

5.1 Stratégie

- **Tests unitaires** des règles de stock, exécutés sur PC avec une base SQLite **en mémoire** (`sqflite_common_ffi`) : `flutter test`.
- **Analyse statique** du code : `flutter analyze`.
- **Recette manuelle** rejouée sur la version web puis sur Android, avec les mêmes résultats attendus.
- **Comparaison visuelle** : site en affichage mobile dans Chrome, à côté de l'application sur émulateur ou téléphone.

Tableau 9 : Tests unitaires de CommandeRepository

Test	Ce qui est vérifié
Ajouter un produit décrémente le stock	Stock de 10 ramené à 7 après l'ajout de 3 unités
Refus d'une quantité supérieure au stock	ErreurMetier levée ; stock inchangé
Refus du même produit deux fois	ErreurMetier levée
Modifier la quantité ajuste le stock	De 3 à 5 : stock 5 ; de 5 à 1 : stock 9
Retirer un produit remet en stock	Le stock revient à 10
Supprimer la commande remet tout en stock	Stock 10 ; commande introuvable
mettreAJour annulé en entier en cas d'erreur	Quantité de 50 refusée ; stock toujours à 7 (rollback)

Par ailleurs, le schéma, les transactions de stock et les huit requêtes du tableau de bord ont été **rejoués sur une base SQLite de test** contenant le jeu de démonstration : les résultats obtenus sont ceux indiqués en 4.6 (6 commandes, 1 095 000 FCFA, 182 500 FCFA de panier moyen, 3 produits en stock faible).

5.2 Recette manuelle

Tableau 10 : Scénario de recette (à rejouer sur le web puis sur Android)

N°	Action	Résultat attendu
1	Créer un compte avec un mot de passe de 7 caractères	« Le mot de passe doit contenir au moins 8 caractères. »
2	Se connecter avec un mauvais mot de passe	« Adresse e-mail ou mot de passe incorrect. »
3	Créer un produit (stock 10) et une commande, y ajouter 3 unités	Stock = 7
4	Passer la quantité à 5, puis demander 99	Stock = 5, puis message de stock insuffisant
5	Retirer le produit, le rajouter, puis supprimer la commande	Stock = 10
6	Fermer complètement l'application et la rouvrir (« Se souvenir de moi » coché)	Utilisateur toujours connecté
7	Ouvrir le tableau de bord après chaque modification	Indicateurs et graphiques à jour
8	Bouton retour Android depuis « Clients », puis depuis le tableau de bord	Retour au tableau de bord, puis sortie de l'application

5.3 Contrôles propres à Android

- Écrans de **360 dp** (petit téléphone), **411 dp** (standard) et **tablette** (≥ 992 dp, sidebar fixe).
- **Rotation** de l'écran et **clavier ouvert** sur chaque formulaire.
- **Sélecteur de date** en français et affichage des montants (« 12 500 FCFA »).
- Contenu de la base inspecté avec le **Database Inspector** d'Android Studio (onglet App Inspection) sur une application lancée en mode debug.

6. Build et déploiement Android



Figure 12 : Chaîne de production de l'application

6.1 Préparation

- Nom affiché : attribut `android:label` du fichier `AndroidManifest.xml` ; version : champ `version` de `pubspec.yaml` (ex. `1.0.0+1`).
- `minSdk = 23` dans `android/app/build.gradle.kts`, exigé par `flutter_secure_storage`.
- Icône de l'application générée avec `flutter_launcher_icons` (Android uniquement).
- Aucune permission n'est nécessaire en production : l'application ne se connecte pas à Internet.

6.2 APK de test ou de remise

```
flutter build apk --release
# -> build/app/outputs/flutter-apk/app-release.apk
flutter build apk --release --split-per-abi    # APK plus légers, un par architecture
flutter install                                  # installation sur l'appareil branché
```

6.3 Signature et publication sur Google Play

1. Générer une clé de signature avec `keytool` et la **conserver hors du projet** (sauvegardée, jamais publiée sur GitHub).
2. Créer `android/key.properties` (mots de passe, alias, chemin de la clé) et le référencer dans le bloc `signingConfigs` de `build.gradle.kts`.
3. Construire le paquet de publication : `flutter build appbundle --release` (fichier `app-release.aab`).
4. Créer l'application dans la Google Play Console, remplir la fiche (description, captures, politique de confidentialité) et publier d'abord en **test interne**.
5. Activer « Play App Signing » : Google conserve la clé finale, la clé locale ne sert qu'à envoyer les mises à jour.

Les conditions de publication (frais d'inscription, période de test imposée aux comptes récents, niveau d'API cible) évoluent : elles sont à vérifier dans la Play Console avant de planifier la publication. Pour la remise du TP, l'**APK** suffit.

7. Écarts, limites et perspectives

7.1 Écarts avec la version web

Tableau 11 : Différences entre la version web et la version mobile

Sujet	Version web	Version mobile	Remarque
Pagination	paginate(10)	Liste complète défilante	Adapté au tactile ; `LIMIT/OFFSET` possible si besoin
Limitation des connexions	5 tentatives par minute	Non reproduite	Pas de serveur à protéger ; à ajouter si nécessaire
Hachage du mot de passe	Hachage Laravel	SHA-256 avec sel	Suffisant pour un usage local pédagogique
Données	Une base MySQL partagée	Une base par téléphone	Conséquence directe de « sans backend »
Photos	Colonnes non utilisées	Colonnes non utilisées	Évolution possible avec image_picker

7.2 Limites

- Les données sont **propres à chaque appareil** et sont perdues si l'application est désinstallée.
- Le hachage SHA-256 salé n'est pas un algorithme lent de type bcrypt ou Argon2 : il ne convient pas à une application de production gérant de vrais comptes.
- Les listes ne sont pas paginées : au-delà de quelques milliers de lignes, une pagination SQL serait préférable.

7.3 Perspectives

- Synchronisation multi-appareils avec **Supabase** ou **Firebase** (la couche repository isole déjà l'accès aux données).
- Authentification déléguée à un service dédié (Firebase Auth) et limitation des tentatives de connexion.
- Gestion des **photos** de clients et de produits, **export** des commandes en PDF.
- Pagination SQL, recherche et filtres sur les listes.

8. Conclusion

Ce projet a permis de transformer une application web Laravel en **application Android autonome**, sans aucun backend. L'architecture en couches (écrans, navigation, état, repositories, persistance) reprend la séparation des responsabilités du MVC, et chaque composant Laravel dispose d'un équivalent Flutter clairement identifié.

Le choix de **SQLite** a permis de conserver le modèle relationnel d'origine, les transactions et les requêtes statistiques du tableau de bord. Les règles de gestion des stocks, qui constituent le cœur métier de l'application, sont centralisées dans les repositories et couvertes par des tests unitaires. L'interface reproduit l'identité visuelle Bootstrap grâce à un thème et à des composants réutilisables, tout en s'adaptant aux spécificités d'Android (menu latéral, bouton retour, clavier).

Les principaux compromis sont la **perte du partage des données** entre utilisateurs et un **hachage de mot de passe** simplifié. Ils sont assumés par la contrainte « sans backend » et pourront être levés par l'ajout d'un service cloud dans une version ultérieure.

Annexe A. Liste des fichiers du projet Flutter

Tableau 12 : Fichiers de l'application et rôle de chacun

Fichier	Rôle
lib/main.dart	Point d'entrée : initialise la base, la session, le thème et la langue française
lib/router.dart	Routes go_router et redirection selon la connexion
lib/core/theme.dart	Palette Bootstrap et thème Material
lib/core/format.dart	Formatage des nombres, montants FCFA et dates
lib/core/ui.dart	toast(), retour(), AlertBox
lib/core/database.dart	Schéma SQLite, données de démonstration, migrations
lib/models/produit.dart	Modèle Produit
lib/models/client.dart	Modèle Client (avec région et département)
lib/models/commande.dart	Modèle Commande et total
lib/models/ligne_commande.dart	Modèle LigneCommande, sous-total, quantité maximale
lib/repositories/auth_repository.dart	Inscription, connexion, session, hachage
lib/repositories/produit_repository.dart	Accès aux produits
lib/repositories/client_repository.dart	Accès aux clients ; ErreurMetier
lib/repositories/commande_repository.dart	Commandes, lignes, règles de stock, transactions
lib/repositories/dashboard_repository.dart	Les 8 requêtes statistiques
lib/providers/auth_provider.dart	État de connexion partagé
lib/widgets/app_shell.dart	Mise en page : sidebar ou Drawer, bouton retour
lib/widgets/auth_card.dart	Carte des écrans de connexion et d'inscription
lib/widgets/bs_button.dart	Bouton Bootstrap
lib/widgets/bs_card.dart	Carte Bootstrap
lib/widgets/bs_table.dart	Tableau Bootstrap
lib/widgets/page_header.dart	En-tête de page
lib/widgets/stat_card.dart	Carte d'indicateur
lib/views/auth/login_page.dart	Écran de connexion
lib/views/auth/register_page.dart	Écran d'inscription
lib/views/dashboard/dashboard_page.dart	Tableau de bord et graphiques
lib/views/produits/produits_page.dart	Liste des produits
lib/views/produits/produit_form_page.dart	Ajout et modification d'un produit
lib/views/clients/clients_page.dart	Liste des clients
lib/views/clients/client_form_page.dart	Ajout et modification d'un client
lib/views/commandes/commandes_page.dart	Liste des commandes
lib/views/commandes/commande_detail_page.dart	Détail, ajout de produit, suppression
lib/views/commandes/commande_form_page.dart	Création et modification d'une commande

Fichier	Rôle
lib/views/commandes/ligne_edit_page.dart	Modification ou retrait d’une ligne
test/commande_repository_test.dart	7 tests unitaires des règles de stock

Annexe B. Commandes utiles

```
# Création du projet (Android uniquement)
flutter create --org com.master.di --platforms=android gestion_commande

# Dépendances
flutter pub add sqflite path provider go_router flutter_secure_storage crypto intl fl_chart
flutter pub add flutter_localizations --sdk=flutter
flutter pub add --dev sqflite_common_ffi

# Qualité et exécution
flutter analyze
flutter test
flutter run

# Build
flutter build apk --release
flutter build appbundle --release
```